GUIA VISUAL 010

COPIAR, MOVER E APAGAR

`mv relatorio.md backups/` — se `backups` não existir, você não moveu nada: renomeou o relatório.

010
GUIAS LINUX

O NOME E O CONTEÚDO SÃO COISAS SEPARADAS

`cp a.txt b.txt`

`a.txt` → `inode 42`

`b.txt` → `inode 77`

Dois nomes, **dois conteúdos**. É a operação cara: lê e escreve tudo, ocupa o dobro, demora conforme o tamanho.

`mv a.txt b.txt`

`b.txt` → `inode 42`

~~`a.txt`~~ `nome removido`

Só a **entrada de diretório** muda. Instantâneo e **atômico** mesmo com 40 GB — entre discos diferentes, porém, vira cópia + remoção.

`rm a.txt`

~~`a.txt`~~ × `inode 42`

`links 1 → 0` → `blocos liberados`

Apaga um **nome**. O conteúdo só é liberado quando nenhum nome **e nenhum processo** apontam para ele — por isso apagar log em uso não devolve espaço.

`MV ORIGEM DESTINO` — O DESTINO EXISTE?

NÃO EXISTE

→ `renomeia para destino`

É o caso que morde: um diretório digitado errado vira um arquivo com nome de diretório, em silêncio.

EXISTE E É DIRETÓRIO

→ `move para destino/origem`

O que você queria. **Termine o destino com barra** — `mv x backups/` — e o caso da esquerda vira erro em vez de surpresa.

EXISTE E É ARQUIVO

→ `sobrescreve sem aviso`

Não há confirmação, não há lixeira. Antídotos: `-i` pergunta, `-n` nunca sobrescreve, `--backup=numbered` guarda o antigo.

AS OPÇÕES QUE MUDAM O RESULTADO

OPÇÃO	VALE PARA	O QUE FAZ
`-i` / `-I`	cp mv rm	Pergunta a cada arquivo / **uma vez** pelo lote
`-n`	cp mv	Nunca sobrescreve, e não pergunta
`-r`	cp rm	Recursivo — obrigatório para diretório
`-a`	cp	**Arquivamento**: `-r` + preserva data, dono, permissão e links
`-u`	cp mv	Só age se a origem for mais nova
`-p`	mkdir	Cria a cadeia toda e **não reclama se já existir**

`cmd > cfg.novo && mv cfg.novo cfg`
Gere ao lado e mova por cima: ninguém jamais lê um arquivo pela metade.

`rmdir vazio`
Só remove diretório vazio — a recusa *é* o recurso de segurança.

`rename -n 's/\.jpeg$/\.jpg/' *.jpeg`
Renomeia em lote; o `-n` simula. Rode sempre com ele antes.

`rsync -av --dry-run orig/ dest/`
Árvore grande, retomável, só o que mudou. **Aqui a barra final muda tudo**: com ela copia o conteúdo, sem ela cria `dest/orig`.

`stat arq` · `touch -d "2020-01-01" arq`
mtime conteúdo · **atime** leitura · **ctime** metadados. O `ctime` não se falsifica pelo `touch`: `ctime` mais novo que `mtime` denuncia carimbo alterado.

Não existe lixeira, e o `rm -rf` não pede licença. Ritual antes de todo comando destrutivo: **1.** `pwd` — onde estou? **2.** `ls -la alvo/` — é isto mesmo? **3.** só então o `rm`. Ou troque o passo 2 por `echo rm -rf "$DIR"/*`: o `echo` imprime a linha **já expandida**, sem executar. Quatro linhas que já destruíram sistemas reais: `rm -rf / caminho/` (o espaço apaga a raiz) · `rm -rf "$DIR"/*` com `$DIR` vazia vira `rm -rf /*` · `cd /alvo ; rm -rf *` se o `cd` falhar (use `&&`) · `rm -rf .*`. O `--preserve-root` é padrão, mas cobre só a raiz literal — `/*` passa.

RESUMO RÁPIDO

- ✓ `cp` duplica conteúdo · `mv` mexe só no nome · `rm` remove um nome.
- ✓ Destino com **barra final** transforma o erro silencioso do `mv` em mensagem.
- ✓ `mkdir -p` é idempotente — obrigatório em script com `set -e`.
- ✓ Backup ou evidência é sempre `cp -a`: sem ele, tudo é carimbado com a data de hoje.
- ✓ `rm -I` (maiúsculo) pergunta **uma vez** pelo lote — o meio-termo que se usa de verdade.

NO KALI

Não adote `alias rm='rm -i'`. Na VM funciona; num shell de máquina-alvo autorizada o alias não existe e o reflexo falha — e `rm -f` ignora o `-i` em silêncio. Prefira `trash-cli` com um alias **novo** (`alias lixo='trash-put'`), deixando o `rm` ser o que ele é. **Coleta de evidência é `cp -a` ou `rsync -a`:** preservar data, dono e permissão faz parte da integridade do material. Para imagem de disco, nem um nem outro — é `dcfldd`, que já vem no Kali e calcula o hash durante a cópia (confira o alvo com `lsblk` antes; Vol 8). Wordlist: descompacte em `/tmp` ou no seu `~` com `gunzip -k`, **nunca dentro de `/usr/share`**, que é território do gerenciador de pacotes.